Fusion ERP Lab Assignment (6B)

Phase 2 — Review, Test and Improve the AI-Generated ELM Module

1. Objective

In Phase 1, you generated ELM implementations using locally deployed code LLMs.

In Phase 2, each pair will select **one of its Phase-1 ELM implementations** and attempt to improve it into a reliable, deployment-ready ELM module that works correctly with the supplied **Fusion Base Code**.

Your task is to:

understand the current state of the generated code, identify its problems, test and review it, and improve it as far as reasonably possible.

Unlike Phase 1, this phase is intentionally open-ended. You are expected to decide what needs to be checked and what engineering actions are required.

2. Inputs

For Phase 2, you will use:

1. **One selected Phase-1 ELM implementation**
2. The corresponding **locally deployed code LLM**
3. The supplied **Fusion Base Code**
4. The complete **ELM specification package**
5. Normal software-development, review, testing, and debugging tools as required

The reason for selecting one of your Phase-1 implementations/code LLMs will be collected separately.

3. Preserve the Phase-1 Implementation

Before starting Phase 2, preserve the selected Phase-1 implementation exactly as it exists.

Do not overwrite or lose the original Phase-1 version.

Phase 2 should begin from that implementation and improve it progressively.

The original Phase-1 code and the final Phase-2 code will later be evaluated separately.

4. Phase-2 Task

Treat the selected Phase-1 implementation as **inherited software that now has to be reviewed, tested, corrected, and prepared for deployment with Fusion ERP**.

A useful way to approach the task is given below.

Step 1 — Understand the Existing Implementation

First inspect the generated code and determine:

- what ELM functionality has been implemented;
- how the frontend and backend are organized;
- how the module uses the Fusion Base;
- what database/data structures have been created;
- which parts appear complete, incomplete, incorrect, or unclear.

Do not assume that generated code is correct simply because it looks reasonable.

Step 2 — Build and Run the System

Attempt to:

- install required dependencies;
- build the frontend and backend;
- run the application;
- initialize/use the database;
- access the ELM module through the Fusion system.

Resolve build, runtime, configuration, or integration problems that prevent meaningful evaluation of the module.

Step 3 — Review and Test the Implementation

Use the complete ELM specification package to determine whether the generated implementation behaves as intended.

You may use any reasonable combination of techniques, including:

- manual code review;
- specification/requirements review;
- manual functional testing;
- unit testing;
- integration testing;
- frontend/backend testing;
- AI-generated tests;
- API testing;
- database inspection;
- static-analysis tools;
- security checks;
- debugging tools;
- logs and runtime inspection.

You are **not required to use every technique**.

Choose the approaches that are useful for the code and problems you encounter.

Step 4 — Identify Important Problems

Look for meaningful issues such as:

- missing functionality;
- incorrect functionality;
- incorrect business rules;
- wrong role/access permissions;
- incorrect workflow/state transitions;
- frontend/backend mismatch;
- Fusion integration problems;
- database/data problems;
- runtime failures;
- insufficient validation;
- poor error handling;
- security concerns;
- unnecessary duplication;
- maintainability or code-quality problems.

Focus on issues that materially affect the correctness, integration, reliability, security, or maintainability of the module.

Step 5 — Improve the Implementation

Use the selected local code LLM together with appropriate software-engineering tools to improve the Phase-1 implementation.

You may:

- ask the LLM to review existing code;
- provide relevant ELM specifications;
- ask it to diagnose failures;
- generate or improve tests;
- implement missing functionality;
- fix functional defects;
- correct business rules and workflows;
- correct authorization/role handling;
- resolve Fusion integration problems;
- improve database handling;
- improve validation and error handling;
- refactor poor-quality code;
- improve security;
- resolve build/runtime failures.

You may interact with the LLM repeatedly as needed.

Phase 2 is not a one-prompt exercise.

Step 6 — Verify the Improvements

After making changes, verify that the changes actually improve the system.

Where applicable:

- rerun the application;
- rerun relevant tests;
- check affected workflows;
- verify role/access behavior;
- verify database changes;
- make sure fixes have not introduced new problems.

Do not consider a problem resolved merely because code was changed.

5. Target Outcome

Your goal is to bring the ELM implementation as close as reasonably possible to a:

deployment-ready ELM module integrated with the Fusion Base

This means that, as far as you can establish:

- the application builds and runs;
- ELM remains correctly integrated with Fusion;
- major required ELM functionality works;
- important business rules are implemented correctly;
- role and access restrictions behave correctly;
- important workflows/state transitions behave correctly;
- obvious functional defects have been addressed;
- reasonable testing/review has been performed;
- major reliability, security, or maintainability problems have been considered;
- no known critical issue remains unresolved.

You are **not expected to guarantee perfect industrial production quality**.

The objective is to make a serious engineering effort to move the Phase-1 generated code toward a reliable, deployable state.

6. Freedom in Phase 2

Phase 2 is intentionally exploratory.

There is **no single prescribed sequence of testing, review, or repair activities**.

Different pairs may discover different problems and may use different approaches to solve them.

You are encouraged to use your engineering judgment in deciding:

- what to inspect first;
- what to test;
- which tools to use;
- when to ask the LLM for help;
- when manual investigation is more appropriate;
- what problems deserve priority;
- when the implementation is sufficiently improved.

This freedom is an important part of the exercise.

7. Do's

- begin from the selected Phase-1 implementation;
- preserve the original Phase-1 version;
- use the complete ELM specifications during review and improvement;
- actively test and inspect the generated code;
- use the selected local code LLM iteratively;
- use suitable non-AI software-engineering tools where helpful;
- investigate why failures occur rather than only patching symptoms;
- verify important fixes;
- pay attention to both functional correctness and Fusion integration;
- consider code quality, reliability, security, and maintainability where relevant;
- make the strongest reasonable attempt to reach a deployment-ready implementation.

8. Don'ts

- replace the Phase-1 code with a completely unrelated ELM implementation obtained elsewhere;
- copy/use another pair's ELM implementation;
- overwrite or lose the original Phase-1 version;
- assume AI-generated code is correct without checking it;
- treat successful compilation alone as proof that the module is correct;
- focus only on UI appearance while ignoring backend behavior, rules, roles, workflows, or data;
- accept an AI-generated fix without verifying its effect;
- ignore known serious problems simply to make the application appear complete.

9. In Short

Phase 1

Generate an ELM implementation integrated with the Fusion Base.

Phase 2

Understand → Build/Run → Review → Test → Identify Problems → Improve → Verify

The central Phase-2 question is:

How far can the Phase-1 AI-generated implementation be improved toward a reliable, deployment-ready Fusion ELM module through systematic human–AI software-engineering effort?